DEVSECOPS DEEP DIVE

GitHub OIDC-Based Secure Deployment Architecture for Amazon EKS

No long-lived secrets. No kubeconfig stored in GitHub. Short-lived OIDC tokens only.

A complete, beginner-friendly reference: what every resource is, why it exists, and how every piece connects to the next.

DevOps Shack | @devopsshack | 2026

Chapter 1: The Big Picture — Why Do Any of This?

Before writing a single AWS CLI command, let's understand the problem we are solving. This context is what makes every subsequent step feel logical rather than arbitrary.

1.1 The Old Way — Static Secrets (and Why It Hurts)

In the traditional CI/CD setup, deploying to Kubernetes from GitHub Actions looks like this:

- You generate a long-lived kubeconfig file or a Kubernetes ServiceAccount token.
- You base64-encode it and paste it into GitHub Secrets as KUBE_CONFIG_B64 or K8S_TOKEN.
- Every pipeline run decodes and uses this token to talk to your cluster.

This works — but it carries serious risks:

Risk	What actually goes wrong
Token never expires	If a static token leaks (e.g. in logs, a public fork, a compromised laptop), it keeps working forever — until someone manually rotates it.
Broad permissions	Kubernetes ServiceAccount tokens are often cluster-admin or have wide namespace access, so a leak = full cluster compromise.
No audit trail	AWS/EKS cannot tell you which pipeline run used a token. It just sees an anonymous bearer token.
Secret sprawl	As team grows, KUBE_CONFIG_B64, K8S_TOKEN, K8S_CA_CERT all sit in GitHub Secrets. Managing rotation is painful.

1.2 The New Way — OIDC Federation (Short-lived, Scoped, Auditable)

OIDC (OpenID Connect) is a standard identity protocol. GitHub has become an OIDC identity provider, meaning it can issue cryptographically-signed certificates that prove: 'This API call is coming from workflow run #1234, in repo jaiswaladi246/3-Tier-NodeJS-End-To-End-Project, on the main branch.'

AWS can verify that certificate independently — no username, no password, no static token involved. The certificate (called a JWT) expires in about 5 minutes. Here is why that is better:

- A leaked token is useless after 5 minutes.
- The IAM role trust policy can be scoped to only one specific repo and branch. A fork cannot assume the same role.
- CloudTrail shows which IAM role was assumed, when, and which job triggered it — full audit trail.

- No credentials are stored in GitHub Secrets. You only store the IAM role ARN (a public identifier, not a secret).

The one-line summary:
GitHub proves its identity cryptographically. AWS verifies the proof and issues 15-minute temporary AWS credentials. Those credentials generate a short-lived kubeconfig. kubectl uses that kubeconfig to deploy. Nothing sensitive is stored anywhere.

1.3 How All the Pieces Connect — The Full Chain

Read this chain before diving into any individual step. Every subsequent chapter is one link in this chain:

1. GitHub Actions workflow runs and requests an OIDC token from GitHub.
2. GitHub issues a signed JWT proving the workflow's identity (repo, branch, ref).
3. The aws-actions/configure-aws-credentials action sends that JWT to AWS STS.
4. AWS STS looks up the registered GitHub OIDC provider and verifies the JWT signature.
5. AWS STS checks the IAM role's trust policy: does this token's audience and subject match the allowed values?
6. If yes, STS issues temporary AWS credentials (access key + secret + session token, valid 15 min).
7. The aws eks update-kubeconfig command uses those credentials to call eks:DescribeCluster and build a kubeconfig.
8. kubectl uses the kubeconfig's exec plugin to get an EKS token, which is verified against the EKS access entry.
9. The access entry maps the IAM role to the AmazonEKSEditPolicy — scoped only to namespace project.
10. kubectl apply deploys the workloads. Job ends, credentials expire, nothing persists.

Chapter 2: Preconditions — What Must Be True Before You Start

Skipping preconditions is the number-one cause of confusing errors mid-setup. This chapter explains each check and why it matters.

2.1 AWS CLI Authentication

Every AWS CLI command in this guide runs as whichever IAM principal is currently authenticated. If you are using the wrong account or an under-privileged role, commands will fail with cryptic AccessDenied errors.

```
# Verify who you are logged in as
aws sts get-caller-identity

# Expected output:
{
  "UserId": "AIDAXXXXXXXXXXXXXXXX",
  "Account": "585593375370",
  "Arn": "arn:aws:iam::585593375370:user/your-admin-user"
}
```

Required permissions for this guide:
Your IAM principal must be able to: create/manage OIDC providers (iam:CreateOpenIDConnectProvider), create IAM roles and policies (iam:CreateRole, iam:AttachRolePolicy), and manage EKS access entries (eks:CreateAccessEntry, eks:AssociateAccessPolicy, eks:UpdateClusterConfig).

2.2 EKS Cluster Must Be ACTIVE

Access entries (the new auth system we will use) only work when the cluster is fully running. If the cluster is CREATING or UPDATING, commands will fail.

```
aws eks describe-cluster \
  --name my-cluster \
  --region ap-south-1 \
  --query
"{status:cluster.status,authMode:cluster.accessConfig.authenticationMode}" \
  --output json

# Expected output:
{
  "status": "ACTIVE",
  "authMode": "API_AND_CONFIG_MAP"
}
```

2.3 Authentication Mode Must Support Access Entries

EKS originally used only the aws-auth ConfigMap to control who could access a cluster. This was error-prone — one bad YAML edit and you could lock yourself out. AWS introduced Access Entries API as a safer alternative.

The catch: access entries only work if the cluster's authentication mode is API or API_AND_CONFIG_MAP. The legacy-only CONFIG_MAP mode does not support access entries at all.

Auth Mode	Supports Access Entries?	What to Do
CONFIG_MAP	No	Must upgrade — see Section 6.1
API	Yes	Ready to use as-is
API_AND_CONFIG_MAP	Yes	Already done — no action needed

Chapter 3: Registering GitHub as a Trusted Identity Provider

This is the foundation of the entire trust chain. Without this step, AWS has no way to verify any token GitHub sends.

3.1 What Is an OIDC Provider (in Plain Language)?

Think of it like this: you want to let people log into your building using their company ID badge. But your security desk does not know that company. You need to go to that company's HR department, verify that their badges are legitimate, and add them to your approved-vendor list.

In our case:

- Your AWS account = the building with a security desk
- GitHub = the company issuing ID badges (JWTs)
- Creating the OIDC provider = adding GitHub to your approved-vendor list in AWS IAM

Specifically, AWS needs three pieces of information to trust GitHub tokens:

11. The provider URL — so AWS knows which endpoint to check the JWT signature against.
12. The audience (client ID) — so AWS knows these tokens were meant for AWS STS, not some other service.
13. The TLS thumbprint — a fingerprint of GitHub's TLS certificate chain, so AWS can verify it is really talking to GitHub, not an impersonator.

3.2 Creating the OIDC Provider

```
# Step 1: Check if the provider already exists
aws iam list-open-id-connect-providers \
  --query
"OpenIDConnectProviderList[?contains(Arn,'token.actions.githubusercontent.com')].A
rn" \
  --output text

# If it exists and you need to recreate it, delete first:
aws iam delete-open-id-connect-provider \
  --open-id-connect-provider-arn arn:aws:iam::585593375370:oidc-
provider/token.actions.githubusercontent.com

# Create the provider:
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

Why sts.amazonaws.com as the client ID?

When GitHub issues the JWT, it sets an 'audience' field in the token. The official aws-actions/configure-aws-credentials action always sets this to sts.amazonaws.com. Your OIDC provider must match this value, otherwise AWS will reject the token with an 'aud claim mismatch' error.

Chapter 4: The IAM Role and Trust Policy — The Core of the Auth Gate

The IAM role is the identity that GitHub Actions will 'become' inside AWS. The trust policy on that role is the security gate that controls exactly who is allowed to become it.

4.1 What Is an IAM Role?

An IAM role is like a job title in an organization. The job title (role) has certain permissions attached (what can be done). Any person (or system) who qualifies can temporarily assume that job title.

Unlike an IAM user, a role has no permanent password or access key. You assume a role temporarily and get short-lived credentials — they automatically expire. That is exactly what we want for CI/CD.

4.2 What Is a Trust Policy?

Every IAM role has two types of policies:

- Trust policy (who can assume this role) — this is what we are writing now.
- Permission policy (what the role can do once assumed) — this is what we attach in Chapter 5.

The trust policy is where the OIDC magic happens. It says: 'I will only allow AssumeRoleWithWebIdentity if the web identity token comes from GitHub, has the right audience, and matches the sub claim for our specific repo and branch.'

4.3 Creating the Trust Policy File

On Windows PowerShell, file encoding matters. The AWS CLI will reject a UTF-16 BOM-encoded file with a confusing JSON parse error. Always write the file with ASCII encoding:

```
# Windows PowerShell — write as ASCII to avoid BOM issues
'{ ... full JSON ... }' | Out-File -FilePath .\github-oidc-trust.json -Encoding
ascii

# The complete trust policy:
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::585593375370:oidc-
provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
```

```
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub":
          "repo:jaiswaladi246/3-Tier-NodeJS-End-To-End-
Project:ref:refs/heads/main"
      }
    }
  }]
}
```

4.4 Dissecting Every Field in the Trust Policy

Every field in this JSON is there for a reason. Here is what each one does:

Field	Purpose and Why It Matters
`Effect: Allow`	Grants permission. Without this, the statement would do nothing. Note: a role with NO trust policy cannot be assumed by anyone — it is locked by default.
`Principal: Federated`	Identifies who is allowed to assume this role. 'Federated' means an external identity provider (GitHub in this case). Using the OIDC provider ARN here means only GitHub-issued tokens are considered — not IAM users, not EC2 instances.
`Action: AssumeRoleWithWebIdentity`	This is the specific STS action that exchanges a web identity token (JWT) for temporary AWS credentials. It is different from AssumeRole (used by IAM users/roles) because it is designed for external identity tokens.
`Condition: aud = sts.amazonaws.com`	Verifies the token's audience field matches exactly. This prevents a JWT issued for some other AWS service or non-AWS service from being misused to assume this role. It is a double-check that the token was intended for STS.
`Condition: sub = repo:...main`	This is the most important security control. The sub (subject) claim in GitHub's JWT encodes the exact repo, ref type, and branch. Using StringLike with this value means only a workflow running on the main branch of your specific repository can assume this role. A fork, a PR branch, or any other repo is rejected.

4.5 Creating the IAM Role

```
# Create the role using the trust policy file
aws iam create-role \
```

```
  --role-name GitHubActionsEKSDeployRole \
  --assume-role-policy-document file://github-oidc-trust.json

# Verify it was created correctly
aws iam get-role --role-name GitHubActionsEKSDeployRole
```

Why is the role called GitHubActionsEKSDeployRole?

The name is descriptive by design. Anyone reading your IAM console immediately understands: this role is for GitHub Actions, and it is specifically for EKS deployments. Good naming hygiene is a security practice — it helps you audit roles and spot unexpected ones.

Chapter 5: The IAM Permission Policy — What the Role Can Do

Having a role (identity) is not enough. You also need to define what that role is allowed to do inside AWS. That is what the permission policy controls. This is the principle of least privilege in action.

5.1 Why Do We Need eks:DescribeCluster?

When your GitHub Actions workflow runs this command:

```
aws eks update-kubeconfig --name my-cluster --region ap-south-1
```

Behind the scenes, the AWS CLI calls the EKS API's DescribeCluster endpoint. This returns:

- The cluster API server endpoint (e.g. https://ABCDEF.gr7.ap-south-1.eks.amazonaws.com)
- The cluster's certificate authority (CA) data
- The cluster name and ARN

The CLI uses this data to build the kubeconfig file. Without eks:DescribeCluster permission, this call is rejected and kubectl cannot be configured at all.

5.2 Why Only eks:DescribeCluster — Not eks:* or AdministratorAccess?

This is least-privilege principle. The deployment role should be able to do exactly one thing in the IAM/EKS control plane: read cluster metadata. It should not be able to:

- Create or delete EKS clusters
- Modify EKS cluster configuration
- Create IAM users or roles
- Access S3, RDS, or any other service

The actual Kubernetes-level permissions (what kubectl can do) are handled separately by EKS access entries in Chapter 6, not by IAM policies. This two-layer design means even if the IAM role is compromised, the damage is limited.

5.3 Creating and Attaching the Policy

```
# Create the minimal policy file
{
  "Version": "2012-10-17",
```

```
  "Statement": [{
    "Sid": "EKSDescribeCluster",
    "Effect": "Allow",
    "Action": ["eks:DescribeCluster"],
    "Resource": "arn:aws:eks:ap-south-1:585593375370:cluster/my-cluster"
  }]
}

# Create the IAM policy in AWS
aws iam create-policy \
  --policy-name GitHubActionsEKSDescribeClusterPolicy \
  --policy-document file://eks-describe-cluster-policy.json

# Attach the policy to the role
aws iam attach-role-policy \
  --role-name GitHubActionsEKSDeployRole \
  --policy-arn
arn:aws:iam::585593375370:policy/GitHubActionsEKSDescribeClusterPolicy

# Verify attachment
aws iam list-attached-role-policies --role-name GitHubActionsEKSDeployRole
```

IAM Policy vs Trust Policy — the key distinction:

The trust policy answers WHO can assume this role. The permission policy answers WHAT this role can do after being assumed. Both are required. A role with a trust policy but no permission policy can be assumed but cannot do anything useful.

Chapter 6: EKS Access Entries — Connecting IAM to Kubernetes RBAC

You now have an IAM role that can be assumed by GitHub Actions and can call eks:DescribeCluster. But that role still cannot talk to the Kubernetes API inside the cluster. EKS access entries bridge this gap.

6.1 What Problem Do Access Entries Solve?

Kubernetes has its own auth system (RBAC — Role-Based Access Control) that is completely separate from AWS IAM. By default, an IAM role — even with full AdministratorAccess — cannot kubectl apply to a cluster.

Historically, the bridge was a ConfigMap called aws-auth in the kube-system namespace. You had to manually add entries like:

```
# Old way (aws-auth ConfigMap) — error-prone YAML editing
mapRoles: |
  - rolearn: arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole
    username: github-actions
    groups:
      - system:masters  # ← dangerously broad
```

Problems with the old way:

- One YAML indentation error could corrupt the ConfigMap and lock everyone out of the cluster.
- No easy CLI — you had to kubectl edit or kubectl apply the ConfigMap manually.
- Hard to audit — who added what entry and when?
- system:masters gives full cluster admin — no namespace scoping.

Access entries solve all of this. They are managed through the AWS API (not YAML), auditable in CloudTrail, and support namespace-scoped policies.

6.2 Upgrading Authentication Mode (if needed)

```
# Check current mode
aws eks describe-cluster \
  --name my-cluster --region ap-south-1 \
  --query "cluster.accessConfig.authenticationMode" --output text

# If it returns CONFIG_MAP, upgrade it:
aws eks update-cluster-config \
  --name my-cluster --region ap-south-1 \
  --access-config authenticationMode=API_AND_CONFIG_MAP

# Wait for cluster to finish updating:
```

```
aws eks describe-cluster \
  --name my-cluster --region ap-south-1 \
  --query "cluster.status" --output text
# Wait until it shows: ACTIVE
```

API_AND_CONFIG_MAP vs API:
We use API_AND_CONFIG_MAP (not just API) because it is the safe migration path. It keeps existing aws-auth entries working while also supporting access entries. Switching directly to API-only would break any existing aws-auth mappings. Use API_AND_CONFIG_MAP during migration, then API once you have fully moved off ConfigMap.

6.3 Creating the Access Entry

An access entry simply registers an IAM principal (our role) as a known identity in the EKS cluster. Think of it as creating a 'known user' record in Kubernetes for our IAM role.

```
# Create the access entry
aws eks create-access-entry \
  --cluster-name my-cluster \
  --region ap-south-1 \
  --principal-arn arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole

# Verify it exists
aws eks describe-access-entry \
  --cluster-name my-cluster \
  --region ap-south-1 \
  --principal-arn arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole
```

6.4 Associating a Namespace-Scoped Access Policy

Creating the access entry only registers the identity. You still need to grant it Kubernetes RBAC permissions. AWS provides managed access policies that map to Kubernetes roles:

AWS Managed Policy	Kubernetes Equivalent	What It Can Do
`AmazonEKSClusterAdminPolicy`	cluster-admin	Full cluster access — avoid for CI/CD
`AmazonEKSAdminPolicy`	admin (namespace)	Full namespace admin
`AmazonEKSEditPolicy`	edit	Create/update/delete most resources (no RBAC)
`AmazonEKSViewPolicy`	view	Read-only access to resources

We use AmazonEKSEditPolicy scoped to namespace project:

```
# Associate the edit policy, scoped to namespace 'project' only
aws eks associate-access-policy \
  --cluster-name my-cluster \
  --region ap-south-1 \
  --principal-arn arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSEditPolicy \
  --access-scope type=namespace,namespaces=project

# Verify the association
aws eks list-associated-access-policies \
  --cluster-name my-cluster \
  --region ap-south-1 \
  --principal-arn arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole
```

What namespace scoping achieves:

GitHub Actions can deploy to namespace 'project' — and ONLY namespace 'project'. It cannot touch kube-system, default, monitoring, or any other namespace. If the IAM role is ever compromised, the blast radius is limited to one namespace. This is defence-in-depth.

Chapter 7: GitHub Repository Setup and the Workflow

With all the AWS infrastructure in place, you now configure GitHub to use it.

7.1 What Secret to Store (and What to Remove)

You only need one GitHub repository secret now:

Secret Name	Value
`AWS_ROLE_TO_ASSUME`	`arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole`

These secrets can now be deleted because they are no longer used:

Secret to Remove	Why It Is No Longer Needed
`KUBE_CONFIG_B64`	Was a static kubeconfig. Now generated fresh on every run via aws eks update-kubeconfig.
`K8S_SERVER`	The cluster endpoint is now read dynamically via eks:DescribeCluster.
`K8S_CA_CERT`	The cluster CA cert is read dynamically from EKS, not stored as a secret.
`K8S_TOKEN`	Was a static ServiceAccount token. Now replaced by 15-minute OIDC-derived tokens.

7.2 The Complete GitHub Actions Workflow

Here is a complete, production-ready workflow file that uses the OIDC pattern:

```
# .github/workflows/deploy.yml

name: Deploy to EKS

on:
  push:
    branches: [main]

permissions:
  id-token: write   # REQUIRED: allows requesting the OIDC JWT
  contents: read    # REQUIRED: allows reading repo contents

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
```

```
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
          aws-region: ap-south-1
          # No access-key-id or secret-access-key needed!

      - name: Configure kubectl
        run: |
          aws eks update-kubeconfig \
            --name my-cluster \
            --region ap-south-1

      - name: Deploy to namespace project
        run: |
          kubectl apply -f k8s/ -n project
          kubectl rollout status deployment/my-app -n project
```

7.3 Docker Registry Secret (One-Time Setup)

If your Kubernetes pods pull images from a private Docker registry, you need an image pull secret in the project namespace. This is a one-time setup step:

```
kubectl create secret docker-registry regcred \
  --docker-username=devopsshack \
  --docker-password=YOUR_DOCKERHUB_PASSWORD \
  --docker-email=devopsshack2025@gmail.com \
  --namespace=project
```

Keep Docker credentials out of your workflow YAML:
The kubectl create secret command should be run once manually (from your local machine or admin account), not in the CI/CD pipeline. The secret lives in Kubernetes and gets used automatically when pods are scheduled. Never put registry passwords in workflow files or GitHub Secrets in plain text.

Chapter 8: Putting It All Together — What Happens When You Push to main

Now that every piece is in place, let us trace exactly what happens from the moment you git push to the moment your pods are running. This chapter is the payoff — every resource we created plays a role here.

8.1 Step-by-Step Runtime Trace

#	Actor	What Happens
1	**Developer**	git push origin main — triggers the GitHub Actions workflow.
2	**GitHub**	Workflow starts. Because permissions: id-token: write is set, GitHub generates a signed JWT containing: iss, sub (repo+branch), aud (sts.amazonaws.com), exp (~5 min).
3	**aws-actions action**	Calls AWS STS AssumeRoleWithWebIdentity, passing the JWT and the role ARN from AWS_ROLE_TO_ASSUME.
4	**AWS STS**	Looks up the OIDC provider (token.actions.githubusercontent.com). Verifies JWT signature using GitHub's public key. Evaluates trust policy conditions (aud and sub). Issues 15-minute temporary credentials.
5	**GitHub Actions runner**	Receives temp credentials: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN. These are set as environment variables — not stored anywhere.
6	**aws eks update-kubeconfig**	Uses temp credentials to call eks:DescribeCluster. Writes kubeconfig to ~/.kube/config with an exec plugin entry (calls aws eks get-token at runtime).
7	**kubectl apply**	kubectl calls the exec plugin → aws eks get-token → returns a 15-min Kubernetes bearer token. Sends kubectl apply request to EKS API server with this token.
8	**EKS API server**	Receives the kubectl request with the bearer token. Validates the token against the access entry for GitHubActionsEKSDeployRole. Checks that the namespace requested (project) is within the allowed scope of AmazonEKSEditPolicy.
9	**EKS (Kubernetes)**	Applies the manifests in namespace project. Pods are scheduled, services created, deployments rolled out.
10	**End of job**	Workflow completes. Temp credentials expire. The kubeconfig is gone (ephemeral runner). Nothing persists. Next run starts fresh.

8.2 Security Properties This Architecture Guarantees

- No static credentials anywhere — nothing to leak, rotate, or accidentally commit.

- Every token expires within 15 minutes — a compromised token has a very short attack window.
- Branch-locked trust — only the main branch can trigger deployments. PRs from forks cannot assume the role.
- Namespace-scoped Kubernetes access — even a fully compromised IAM role cannot touch other namespaces or cluster-level resources.
- Full audit trail — CloudTrail records every STS AssumeRole event with the JWT subject (repo + branch), giving you per-commit, per-run audit logs.

Chapter 9: Quick Reference — All Commands in Order

Use this chapter as a cheat sheet. Run these commands in order on a fresh setup.

Step 1 — Verify prerequisites

```
aws sts get-caller-identity
aws eks describe-cluster --name my-cluster --region ap-south-1 \
  --query
"{status:cluster.status,authMode:cluster.accessConfig.authenticationMode}"
```

Step 2 — Create OIDC provider

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

Step 3 — Create IAM role

```
# Write trust policy as ASCII (Windows PowerShell):
'{ ... }' | Out-File -FilePath .\github-oidc-trust.json -Encoding ascii

aws iam create-role \
  --role-name GitHubActionsEKSDeployRole \
  --assume-role-policy-document file://github-oidc-trust.json
```

Step 4 — Create and attach IAM policy

```
aws iam create-policy \
  --policy-name GitHubActionsEKSDescribeClusterPolicy \
  --policy-document file://eks-describe-cluster-policy.json

aws iam attach-role-policy \
  --role-name GitHubActionsEKSDeployRole \
  --policy-arn
arn:aws:iam::585593375370:policy/GitHubActionsEKSDescribeClusterPolicy
```

Step 5 — Enable access entries (if needed)

```
aws eks update-cluster-config \
  --name my-cluster --region ap-south-1 \
  --access-config authenticationMode=API_AND_CONFIG_MAP
```

Step 6 — Create access entry and associate policy

```
aws eks create-access-entry \
  --cluster-name my-cluster --region ap-south-1 \
  --principal-arn arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole

aws eks associate-access-policy \
  --cluster-name my-cluster --region ap-south-1 \
  --principal-arn arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSEditPolicy \
  --access-scope type=namespace,namespaces=project
```

Step 7 — Add GitHub secret

```
# In GitHub repo Settings > Secrets and variables > Actions:
# Name:  AWS_ROLE_TO_ASSUME
# Value: arn:aws:iam::585593375370:role/GitHubActionsEKSDeployRole
```

Step 8 — Add workflow file and deploy

```
# Add .github/workflows/deploy.yml with the workflow from Chapter 7
git add .github/workflows/deploy.yml
git commit -m 'feat: add OIDC-based EKS deployment workflow'
git push origin main
```